1. Introduction

Most snippets assume the following macros.

```cpp
#include <bits/stdc++.h>

using namespace std;

#define P push_back
#define I insert
#define X extract
#define L lower_bound
#define U upper_bound
#define C count

auto ckmn = [](auto &x, auto y) { return y < x ? x = y, true : false; };
auto ckmx = [](auto &x, auto y) { return y > x ? x = y, true : false; };
auto lso  = [](integral auto n) { return n & (-n); };
auto pop  = [](integral auto n) { return popcount((unsigned long long)n); };

#define W(x, y) ((x & y) == y)
#define N(i, n) " \n"[i == n]
#define Z(x, i) (x >> i & 1)
#define F(i, l, r) for (int i = l; i < r; i++)
#define G(i, l, r) for (int i = l; i ⩽ r; i++)
#define H(i, r, l) for (int i = r; i ⩾ l; i--)
#define A(a) (a).begin(), (a).end()
#define R(a) (a).rbegin(), (a).rend()
#define S(a) (int)ssize(a)
#define B(a) (a).begin()
#define E(a) (a).end()
#define T(a) prev(E(a))
#define O(a) sort(A(a)), a.erase(unique(A(a)), E(a))

template <typename T = int>
using V  = vector<T>;
using ll = long long;
using ul = unsigned long long;
using ii = array<int, 2>;

constexpr int oo = 0x3f3f3f3f;
constexpr ll OO  = 0x3f3f3f3f3f3f3f3f;
```

1.1. Template (Proton)

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<ll> vll;
typedef vector<vll> vvll;
typedef pair<ll, ll> pll;
typedef vector<pll> vpll;
typedef vector<int> vi;
typedef vector<vi> vvi;
typedef pair<int, int> pi;
typedef vector<pi> vpi;
typedef set<ll> sll;
typedef set<int> si;
typedef map<ll, ll> mll;
typedef map<int, int> mi;
typedef vector<bool> vb;
#define pb push_back
#define INF(dt) numeric_limits<dt>::max()
#define NINF(dt) numeric_limits<dt>::min()
```

1.2. Template (Baytoro)

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define pb push_back
#define fr first
#define sc second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define int ll
```

1.3. I/O

For large inputs, it may be necessary to enable fast I/O.

```cpp
cin.tie(nullptr)->sync_with_stdio(false);
```

Regarding interactive problems, remember to flush the stream using `endl`!

1.4. Order Statistic Tree

```cpp
#include <bits/extc++.h>

using namespace std;

using Ost = __gnu_pbds::tree<
    int, __gnu_pbds::null_type, less<>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>;
```

1.5. Custom Hash

While this slows down `unordered_map` a little, it helps prevent hacks.

```cpp
struct custom_hash { // PERF: https://codeforces.com/blog/entry/62393
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x^(x>>30)) * 0xbf58476d1ce4e5b9;
        x = (x^(x>>27)) * 0x94d049bb133111eb;
        return x^(x>>31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
```

1.6. Random Number Generation

To generate a random integer, first set up `gen`.

```cpp
mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
```

After which, you can use `uniform_int_distribution` to produce a random integer in a closed interval.

2. Disjoint Set Union

Nodes with negative values are roots. `cmpCnt` and `cmpMax` find the number of components and the size of the largest component respectively. Omit if not necessary.

```cpp
struct DSU {
  int cmpCnt, cmpMax;
  V<int> nodes; // WARN: assumes `nodes` is zero-indexed
  DSU(int n)
      : cmpCnt(n)
      , cmpMax(1)
      , nodes(n, -1) {}
  int find(int x) { return nodes[x] < 0 ? x : nodes[x] = find(nodes[x]); }
  int size(int x) { return -nodes[find(x)]; }
  bool unite(int x, int y) { // WARN: merge `y` into `x`
    x = find(x), y = find(y);
    if (x == y) return false;
    if (nodes[x] > nodes[y]) swap(x, y);
    nodes[x] += nodes[y], nodes[y] = x;
    cmpCnt--;
    ckmx(cmpMax, -nodes[x]);
    return true;
  }
};
```

2.1. Persistent Variant

Check if two nodes are connected at some point in time.

```cpp
struct DSU {
  V<int> size, parent, time_changed; // WARN: assumes `parent` is zero-indexed
  DSU(int n)
      : size(n, 1)
      , parent(n, 0)
      , time_changed(n, 0) {
    iota(A(parent), 0);
  }
  int find(int x, int time) {
    if (parent[x] == x || time_changed[x] > time) return x;
    return find(parent[x], time);
  }
  bool unite(int x, int y, int time) { // WARN: merge `y` into `x`
    x = find(x, time), y = find(y, time);
    if (x == y) return false;
    if (size[x] < size[y]) swap(x, y);
    size[x] += size[y], parent[y] = x, time_changed[y] = time;
    return true;
  }
};
```

3. Sorting

3.1. Custom `struct`/`class`

Suppose we have some custom `struct` representing fractions, to create a `set`, we need to perform operator overloading of `<`.

```cpp
struct Frac {
  int x, y;
  Frac(int x, int y)
      : x(x)
      , y(y) {}
  bool operator<(Frac other) const { return x * other.y < y * other.x; }
};
```

3.2. Inversions Count

An array's *inversions count* refers to the number of adjacent swaps required to sort an array. It can be computed via a modified merge sort, which has the side effect of sorting the array.

```cpp
auto merge = [](auto self, V<int> &v, int lb, int rb) -> ll {
  if (lb == rb) return 0;
  int mb = (lb + rb) / 2, lp = lb, rp = mb + 1;
  ll ans = self(self, v, lb, mb) + self(self, v, mb + 1, rb);
  while (lp ⩽ mb && rp ⩽ rb) {
    if (v[lp] ⩽ v[rp]) lp++;
    else ans += mb - lp + 1, rp++;
  }
  inplace_merge(B(v) + lb, B(v) + mb + 1, B(v) + rb + 1);
  return ans;
};
```

4. Matching

4.1. Maximum Cardinality Bipartite Matching (MCBM)

A maximum matching contains as many edges of a graph G as possible such that each vertex is part of at most one edge of the subset. Assume that `0 ≤ u < lsize` and `0 ≤ v < rsize` in the following implementation.

```
int l_size, r_size, edge_cnt;
mt19937 gen(chrono::steady_clock::now().time_since_epoch().count());

int main() {
  cin >> l_size >> r_size >> edge_cnt;
  V<ii> edg, matching;

  // WARN: assumes both `u` and `v` are zero-indexed
  for (int i = 0, u, v; i < edge_cnt; i++) {
    cin >> u >> v;
    edg.push_back({ u, v });
  }

  shuffle(A(edg), gen);

  auto hopcroft_karp = [&]() -> void {
    V<int> deg(l_size + 1);
    for (auto &[u, v] : edg) deg[u]++;
    partial_sum(begin(deg), end(deg), begin(deg));
    V<int> g(edge_cnt), lmc(l_size, -1), rmc(r_size, -1), a, p,
q(l_size);
    for (auto &[u, v] : edg) g[--deg[u]] = v;
    while (true) {
      a.assign(l_size, -1), p.assign(l_size, -1);
      int t = 0, match = false;
      for (int i = 0; i < l_size; i++) {
        if (lmc[i] == -1) q[t++] = a[i] = p[i] = i;
      }
      for (int i = 0; i < t; i++) {
        int x = q[i];
        if (~lmc[a[x]]) continue;
        for (int j = deg[x]; j < deg[x + 1]; j++) {
          int y = g[j];
          if (rmc[y] == -1) {
            while (~y) {
              rmc[y] = x, swap(lmc[x], y), x = p[x];
            }
            match = true;
            break;
          }
          if (p[rmc[y]] == -1) {
            q[t++] = y = rmc[y], p[y] = x, a[y] = a[x];
          }
        }
      }
      if (!match) break;
    }
    for (int i = 0; i < l_size; i++) {
      if (~lmc[i]) matching.push_back({ i, lmc[i] });
    }
  };
}
```

4.1.1. Minimum Vertex Cover

Assuming you have obtained the matching above, you can retrieve the *minimum vertex cover* (the smallest set of vertices that includes at least one endpoint of every edge of the graph) by constructing a new *directed* graph. Edges that belong to the matching will go from right to left, whereas all other edges will go from left to right. Perform a depth first search starting at all left vertices that are not incident to any edges in the matching.

The minimum vertex cover includes all visited right vertices of the matching, as well as all unvisited left vertices of the matching.

Note that some problems also have situations where there are isolated nodes (degree zero) that have to be taken.

```
int l_size, r_size;
V<ii> matching, edg;
V<int> lhs, rhs;

int main() {
  set<ii> seen;
  set<ii> matching_set(A(matching));

  map<ii, V<ii>> adj;
  set<int> to_start;
  V<int> min_cover;

  F(i, 0, l_size) to_start.I(i);

  for (auto [u, v] : edg) {
    if (matching_set.C({ u, v })) {
      adj[{ v, 1 }].P({ u, 0 });
      to_start.X(u);
    } else {
      adj[{ u, 0 }].P({ v, 1 });
    }
  }

  auto dfs = [&](auto self, int u, int state) -> void {
    seen.I({ u, state });
    for (auto [v, nstate] : adj[{ u, state }]) {
      if (seen.C({ v, nstate })) continue;
      self(self, v, nstate);
    }
  };

  for (auto u : to_start) {
    if (seen.C({ u, 0 })) continue;
    dfs(dfs, u, 0);
  }

  for (auto [u, v] : matching) {
    if (seen.C({ v, 1 })) {
      min_cover.P(rhs[v]);
    }
    if (!seen.C({ u, 0 })) {
      min_cover.P(lhs[u]);
    }
  }
}
```

4.2. Perfect Matching

A perfect matching is one where every vertex is included in exactly one edge of the matching. The following implementation finds the minimum weight perfect matching. To obtain the maximum weight variant, simply invert all the costs.

```
template <typename T>
class Hungarian {
  const T INF = numeric_limits<T>::max();

public:
  T tot = 0;
  vector<int> p, way, ans;
  vector<T> u, v;
  Hungarian(const vector<vector<T>> &a) { // WARN: assumes `a` is
one-indexed
    int n = ssize(a) - 1, m = ssize(a[0]) - 1;
    u.assign(n + 1, 0), v.assign(m + 1, 0);
    p.assign(m + 1, 0), way.assign(m + 1, 0), ans.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) {
      p[0]  = i;
      int j0 = 0;
      vector<bool> usd(m + 1);
      vector<T> mnv(m + 1, INF);
      do {
        usd[j0] = true;
        int i0 = p[j0], j1 = 0;
        T dlt = INF;
        for (int j = 1; j <= m; j++) {
          if (!usd[j]) {
            T cur = a[i0][j] - u[i0] - v[j];
            if (cur < mnv[j]) mnv[j] = cur, way[j] = j0;
            if (mnv[j] < dlt) dlt = mnv[j], j1 = j;
          }
        }
        for (int j = 0; j <= m; j++) {
          if (usd[j]) u[p[j]] += dlt, v[j] -= dlt;
          else mnv[j] -= dlt;
        }
        j0 = j1;
      } while (p[j0]);
      do {
        int j1 = way[j0];
        p[j0]  = p[j1];
        j0     = j1;
      } while (j0);
    }
    for (int j = 1; j <= m; j++) {
      ans[p[j]] = j;
    }
    tot = -v[0];
  }
};
```

5. Square Root Decomposition

Assuming `block` represents the size of a chunk, `a` represents the original, full-sized array, and `b` represents the values of each block, the example snippet demonstrates a sum query. A good value for `block` is around 250.

```
// WARN: assumes zero-indexed queries
auto query = [](int l, int r) -> ll {
  ll ans = 0;
  int bl = l / block, br = r / block;
  if (bl == br) return reduce(begin(a) + l, begin(a) + r + 1);
  for (int i = l; i <= (bl + 1) * block - 1; i++) ans += a[i];
  for (int i = bl + 1; i <= br - 1; i++) ans += b[i];
  for (int i = br * block; i <= r; i++) ans += a[i];
  return ans;
};
```

5.1. Mo's Algorithm

```
// WARN: assumes zero-indexed queries
struct Query {
  int i, l, r;
  Query(int i, int l, int r)
      : i(i)
      , l(l)
      , r(r) {}
  friend bool operator<(const Query &qa, const Query &qb) {
    if (qa.l / block == qb.l / block) return qa.r < qb.r;
    return qa.l < qb.l;
  }
};
V<Query> queries;

void solve() {
  auto ins = [&](int i) {};
  auto del = [&](int i) {};

  int cl = 0, cr = -1, cur = 0;

  for (auto [i, l, r] : queries) {
    while (cl > l) ins(--cl);
    while (cr < r) ins(++cr);
    while (cl < l) del(cl++);
```

```
    while (cr > r) del(cr--);
    ans[i] = cur;
  }
}
```

6. Treap

6.1. Implicit Variant

Insertion and deletion are done based on the zero-indexed position of an element within the tree. Nodes in the snippet store `string` values, remember to modify depending on the problem.

```
mt19937_64
gen(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(0, oo);

struct Node {
  string val;
  int pri, sze;
  Node *lp, *rp;
  Node(string val)
      : val(val)
      , pri(dist(gen))
      , sze(1)
      , lp(nullptr)
      , rp(nullptr) {}
};

int size(Node *root) { return root ? root->sze : 0; }

void split(Node *root, Node *&lt, Node *&rt, int idx) {
  if (!root) {
    lt = rt = nullptr;
    return;
  }
  if (size(root->lp) + 1 <= idx) {
    split(root->rp, root->rp, rt, idx - (size(root->lp) + 1));
    lt = root;
  } else {
    split(root->lp, lt, root->lp, idx);
    rt = root;
  }
  root->sze = 1 + size(root->lp) + size(root->rp);
}

void merge(Node *&root, Node *lt, Node *rt) {
  if (!lt || !rt) {
    root = lt ? lt : rt;
    return;
  }
  if (lt->pri > rt->pri) {
    merge(lt->rp, lt->rp, rt);
    root = lt;
  } else {
    merge(rt->lp, lt, rt->lp);
    root = rt;
  }
  root->sze = 1 + size(root->lp) + size(root->rp);
}

void concat(Node *root, vector<string> &ans) {
  if (!root) return;
  concat(root->lp, ans);
  ans.push_back(root->val);
  concat(root->rp, ans);
}

void clean(Node *root) {
  if (!root) return;
  clean(root->lp);
  clean(root->rp);
  delete root;
}
```

```
}

int n, q, find_idx = -1;

void find(Node *root, string &s, int offset) {
  if (!root) return;

  if (root->val == s) {
    int cur = offset;

    if (root->lp) cur += root->lp->sze;
    find_idx = cur;

    return;
  }

  int inc = 1;
  if (root->lp) inc += root->lp->sze;

  find(root->lp, s, offset);
  find(root->rp, s, offset + inc);
}

void solve_t() {
  cin >> n >> q;

  vector<string> a(n);

  F(i, 0, n) cin >> a[i];

  Node *root = new Node(a[0]);

  auto insert = [&](int i, string s) -> void {
    Node *lp, *rp;
    split(root, lp, rp, i);
    Node *n_node = new Node(s);
    merge(lp, lp, n_node);
    merge(lp, lp, rp);
    root = lp;
  };

  auto erase = [&](int i) -> void {
    Node *lp, *rp, *n_lp, *n_rp;

    split(root, lp, rp, i);
    split(rp, n_lp, n_rp, 1);

    V<string> ans;

    merge(lp, lp, n_rp);
    root = lp;
  };

  clean(root);
}
```

6.2. Cartesian Tree

We may also construct it from an array, assuming no further updates to the array are required. The smallest value belongs to the `root`, which stores the corresponding index. Assuming all values are distinct, the resulting tree is unique.

```
template <typename T>
class CartesianTree {
  struct Node {
    int p, l, r;
    Node(int p = 0, int l = 0, int r = 0)
        : p(p)
        , l(l)
        , r(r) {}
  };

public:
  int root;
  deque<Node> tree;
  CartesianTree(const vector<T> &a)
      : tree(S(a)) {
    for (int i = 1; i <= S(a) - 1; i++) { // WARN: `a` is one-indexed
      tree[i].p = i - 1;
      while (a[i] < a[tree[i].p]) tree[i].p = tree[tree[i].p].p;
      tree[i].l          = tree[tree[i].p].r;
      tree[tree[i].l].p = i;
      tree[tree[i].p].r = i;
    }
    for (int i = 1; i <= S(a) - 1; i++)
      if (tree[i].p == 0) root = i;
  }
};
```

6.3. Lazy Propagation

The following snippet illustrates a non-implicit treap with range addition.

```
mt19937 gen(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(0, INT_MAX);

struct Node {
  int val, pri, add, q; // WARN: rmb to tweak (may not need `q`)
  Node *lp, *rp;
  Node(int val, int q)
      : val(val)
      , pri(dist(gen))
      , add(0)
      , q(q)
      , lp(0)
      , rp(0) {}
};

void push(Node *root) {
  if (!root) return;
  if (root->add) {
    if (root->lp) root->lp->val += root->add, root->lp->add += root->add;
    if (root->rp) root->rp->val += root->add, root->rp->add += root->add;
    root->add = 0;
  }
}

void split(Node *root, Node *&lt, Node *&rt, int x) {
  if (!root) {
    lt = rt = nullptr;
    return;
  }
  push(root), push(lt), push(rt);
  if (root->val <= x) {
    split(root->rp, root->rp, rt, x);
    lt = root;
  } else {
    split(root->lp, lt, root->lp, x);
    rt = root;
  }
}

void merge(Node *&root, Node *lt, Node *rt) {
  push(root), push(lt), push(rt);
  if (!lt || !rt) {
    root = lt ? lt : rt;
    return;
  }
  if (lt->pri >= rt->pri) {
    merge(lt->rp, lt->rp, rt);
    root = lt;
  } else {
    merge(rt->lp, lt, rt->lp);
    root = rt;
  }
}
```

```
}

void range_add(Node *&root, int l, int r, int addend) {
  Node *lt = nullptr, *lm = nullptr, *mt = nullptr, *rt = nullptr;
  split(root, lm, rt, r);
  split(lm, lt, mt, l - 1);
  if (mt) mt->val += addend, mt->add += addend;
  merge(lm, lt, mt);
  merge(root, lm, rt);
}

void insert(Node *&root, int x, int q) {
  push(root);
  Node *lt = nullptr, *rt = nullptr;
  split(root, lt, rt, x);
  merge(lt, lt, new Node(x, q));
  merge(root, lt, rt);
}

void concat(Node *root, vector<ii> &a) {
  if (!root) return;
  push(root);
  concat(root->lp, a);
  a.push_back({ root->q, root->val });
  concat(root->rp, a);
}

void clean(Node *root) {
  if (!root) return;
  clean(root->lp);
  clean(root->rp);
  delete root;
}
```

7. Graphs

7.1. Cycle Retrieval

In problems where `a[i]` represents the next "jump" from position `i`, the following snippet finds all cycles, which may be used for further processing.

```
int main() {
  V seen(n + 1), a(n + 1);

  G(i, 1, n) cin >> a[i];

  // WARN: assumes one-indexed `a`
  for (int i = 1, j, cycs; i <= n; i++) {
    if (seen[i]) continue;
    j = i, cycs = 0;
    set<int> curs;
    deque<int> path;
    do {
      seen[j] = true, curs.insert(j), path.push_back(j);
      j       = a[j];
      if (curs.contains(j)) {
        cycs = j;
        break;
      }
    } while (!seen[j]);
    if (cycs) {
      while (path.front() != cycs) path.pop_front();
    }
  }
}
```

7.2. Diameter

The furthest distance a node `u` is from some other node in a tree is equivalent to `max(dist(dia_u, u), dist(dia_v, v))`, where `dia_u` and `dia_v` represent the endpoints of some diameter.

```
// WARN: assumes zero-indexed `adj`
auto build = [](vector<vector<int>> &adj) -> pair<int, vector<int>>
{
  int init = 0, n = S(adj);
  V<int> df(n, 0), du(n, 0), dv(n, 0);

  auto dfs
      = [&](auto &&self, int u, int pu, vector<int> &d, int cur = 0)
-> void {
    d[u] = cur;
    for (auto v : adj[u]) {
      if (v == pu) continue;
      self(self, v, u, d, cur + 1);
    }
  };

  dfs(dfs, init, -1, df);
  int dia_u = max_element(A(df)) - begin(df);

  dfs(dfs, dia_u, -1, du);
  int dia_v = max_element(A(du)) - begin(du);

  dfs(dfs, dia_v, -1, dv);
  vector<int> ans(n, 0);

  F(i, 0, n) { ckmx(ans[i], max(du[i], dv[i])); }

  return { *max_element(A(du)), ans };
};
```

7.3. Bridges

A *bridge* is an edge of a graph whose deletion increases the graph's number of connected components. `memo[u]` represents the number of *back-edges* passing over the edge between `u` and its parent.

```
int n;

int main() {
  cin >> n;

  V adj(n, V());
  vector<ii> bridges;
  vector<int> depth(n), memo(n);

  // NOTE: assumes zero-indexed nodes
  auto solve = [&](auto self, int u, int pu) -> void {
    for (auto &v : adj[u])
      if (v != pu) {
        if (!depth[v]) {
          depth[v] = depth[u] + 1;
          self(self, v, u);
          memo[u] += memo[v];
        } else if (depth[v] < depth[u]) {
          memo[u]++;
        } else {
          memo[u]--;
        }
      }
    if (depth[u] != 1 && !memo[u]) bridges.push_back({ pu, u });
  };

  depth[0] = 1;
  solve(solve, 0, -1);
}
```

7.4. Cut Vertices

When a *cut vertex* is removed, the graph is disconnected.

```
int n;

int main() {
  cin >> n;

  V adj(n, V());
  vector<int> depth(n), memo(n, INT_MAX), cuts(n, false);

  // NOTE: assumes zero-indexed nodes
  auto solve = [&](auto self, int u, int pu) -> void {
    int children = 0;
    for (auto &v : adj[u])
      if (v != pu) {
        if (!depth[v]) {
          depth[v] = depth[u] + 1;
          self(self, v, u);
          memo[u] = min(memo[u], memo[v]);
          children++;
        } else if (depth[v] < depth[u]) {
          memo[u] = min(memo[u], depth[v]);
        } else {
          memo[v] = min(memo[v], depth[u]);
        }
      }
    if (pu == -1) {
      cuts[u] = (children >= 2);
    } else {
      if (memo[u] >= depth[pu]) cuts[pu] = true;
    }
  };

  depth[0] = 1;
  solve(solve, 0, -1);
}
```

7.5. Edge Cactus

Every edge belongs to at most one cycle. You may find all cycles by merging the edges with `DSU`.

```
int n, m;

int main() {
  cin >> n >> m;

  V adj(n + 1, V<ii>());
  V<ii> edg;

  F(i, 0, m) {
    int u, v;
    cin >> u >> v;
    adj[u].P({ v, i });
    adj[v].P({ u, i });
    edg.P({ u, v });
  };

  V mnb(n + 1, n + 1), depth(n + 1, 0);
  mnb[n] = n;
  DSU dsu(m);

  auto solve = [&](auto &&self, int u, int pu, int ei) -> ii {
    depth[u] = depth[pu] + 1;
    ii dmn   = { depth[u], ei };
    for (auto [v, en] : adj[u]) {
      if (v == pu) continue;
      if (depth[v]) ckmn(dmn, ii{ depth[v], en });
      else ckmn(dmn, (ii)self(self, v, u, en));
    }
    if (dmn[0] != depth[u]) {
      int en = dmn[1];
      dsu.unite(ei, en);
    }
    return dmn;
  };

  G(i, 1, n) {
    if (depth[i]) continue;
```

```
    solve(solve, i, 0, -1);
  }
}
```

7.6. Eulerian Paths/Circuits

7.6.1. Undirected

Assuming we have an undirected *connected* graph, if all vertices have an *even* degree, an Eulerian circuit exists; otherwise, if we have two vertices of odd degree, an Eulerian path exists (`o1` and `o2` represent the two odd vertices).

Be careful about self-loops, they don't count towards the degree! In general this algorithm is pretty expensive, so perform it *just once* if possible.

```
int odds = 0, n, m, o1, o2;
V adj(n + 1, set<int>());
V p;

int main() {
  // NOTE: do some pre-processing
  // ...

  if (odds == 1 || odds >= 3) {
    cout << "IMPOSSIBLE" << '\n';
    return 0;
  }

  auto solve = [&](auto self, int u) -> void {
    auto &vs = adj[u];
    while (!empty(vs)) {
      int v = *begin(vs);
      vs.erase(v);
      adj[v].erase(u);
      self(self, v);
    }
    p.push_back(u);
  };
  // WARN: assume one-indexed nodes
  solve(solve, odds == 2 ? o1 : 1);

  if (S(p) != m + 1) {
    cout << "IMPOSSIBLE" << '\n';
    return 0;
  }
  for (int i = 0; i < m + 1; i++) cout << p[i] << " \n"[i == m];
}
```

7.6.2. Directed

A directed graph has an Eulerian circuit if and only if it is *connected* and each vertex has the same in-degree as out-degree. A directed graph has an Eulerian path if and only if it is connected and each vertex except two has an in-degree as out-degree.

One of the two should have an out-degree one greater than in-degree (*start vertex*), while the other has an in-degree one greater than out-degree (*end vertex*). The core algorithm is the same as that for the undirected graph.

7.7. Binary Lifting

Before we can perform binary lifting, we have to first construct a table `tab`, where `tab[i][h]` represents the 2^h-th ancestor of `i`.

```
int n;

int main() {
  // WARN: assumes zero-indexed nodes
  V adj(n, V());

  auto ceil_log2 = [](int n) -> int {
    int ans = 0, cur = 1;
    while (cur < n) cur *= 2, ans++;
    return ans;
  };

  int tab_height = ceil_log2(n);
  vector depth(n, 0);
  vector tab(n, vector(tab_height + 1, -1));

  auto build_tab = [&](auto self, int u, int pu) -> void {
    tab[u][0] = pu;
    for (auto &v : adj[u])
      if (v != pu) {
        depth[v] = depth[u] + 1;
        self(self, v, u);
      }
  };
  depth[0] = 1;
  build_tab(build_tab, 0, -1);

  for (int k = 1; k <= tab_height; k++) {
    for (int u = 0; u < n; u++) {
      if (tab[u][k - 1] != -1) {
        tab[u][k] = tab[tab[u][k - 1]][k - 1];
      }
    }
  }

  auto get_parent = [&](int u, int k) -> int {
    for (int h = 0; h <= tab_height; h++)
      if ((k >> h) & 1) {
        u = tab[u][h];
        if (u == -1) break;
      }
    return u;
  };

  auto get_lca = [&](auto self, int u, int v) -> int {
    if (depth[u] < depth[v]) return self(v, u);
    u = get_parent(u, depth[u] - depth[v]);
    if (u == v) return u;
    for (int h = tab_height; h >= 0; h--) {
      if (tab[u][h] != tab[v][h]) u = tab[u][h], v = tab[v][h];
    }
    return tab[u][0];
  };

  auto get_dist = [&](auto self, int u, int v) -> int {
    if (depth[u] < depth[v]) return self(v, u);
    int delta = depth[u] - depth[v], ans = delta;
    u = get_parent(u, delta);
    if (u == v) return delta;
    for (int h = tab_height; h >= 0; h--) {
      if (tab[u][h] != tab[v][h]) {
        u = tab[u][h], v = tab[v][h];
        ans += 2 * (1 << h);
      }
    }
    return ans + 2;
  };
}
```

7.8. Heavy Light Decomposition

If you want to update the value of node `u`, you need to update `pos[u]`. `query` needs to be tweaked depending on the problem.

```
int n;
constexpr int NMAX = 100'000;

int main() {
  // WARN: assumes zero-indexed nodes
  V adj(n, V());

  V raw(n), s(n); // NOTE: `s` contains the value in each node
  array<int, NMAX> par{}, depth{}, heavy{}, head{}, pos{};
  fill(A(heavy), -1);
  int cur_pos = 0;

  auto dfs = [&](auto self, int u) -> int {
    int subtree = 1, max_child = 0;
    for (auto &v : adj[u])
      if (par[u] != v) {
        par[v] = u, depth[v] = depth[u] + 1;
        int child = self(self, v);
        if (child > max_child) max_child = child, heavy[u] = v;
      }
    return subtree;
  };
  dfs(dfs, 0);

  auto dc = [&](auto self, int u, int h) -> void {
    head[u] = h, pos[u] = cur_pos, raw[cur_pos] = s[u], cur_pos++;
    if (heavy[u] != -1) self(self, heavy[u], h);
    for (auto &v : adj[u])
      if (par[u] != v && v != heavy[u]) {
        self(self, v, v);
      }
  };
  dc(dc, 0, 0);

  SegTree tree(raw);
  auto query = [&](int u, int v) -> int {
    int ans = numeric_limits<int>::min();
    for (; head[u] != head[v]; v = par[head[v]]) {
      if (depth[head[u]] > depth[head[v]]) swap(u, v);
      ans = tree.conquer(ans, tree.query(pos[head[v]], pos[v]));
    }
    if (depth[u] > depth[v]) swap(u, v);
    return tree.conquer(ans, tree.query(pos[u], pos[v]));
  };
}
```

7.9. Tree Hashing

You can hash a *rooted* tree like so. If it isn't rooted, you may find a centroid first before using it as a root to perform the hash. Some trees may have *two* centroids, in which case you have to retrieve two hashes. Among the two, put the smaller one before the larger one in a pair of integers (e.g., in `V<int>`).

```
auto hash = [](vector<vector<int>> &adj) -> array<Mint, 2> {
  vector subs(n + 1, Mint());

  // WARN: assumes one-indexed nodes
  auto solve
      = [&](auto self, int b = 31, int u = 1, int pu = 0, int d = 0)
-> Mint {
    deque<Mint> vs;
    subs[u]++;

    for (auto v : adj[u]) {
      if (v == pu) continue;
      vs.P(subs[v] * self(self, b, v, u, d + 1));
      subs[u] += subs[v];
    }
    sort(A(vs), [&](Mint &x, Mint &y) { return x.v < y.v; });

    vs.push_front(d), vs.push_back(d);

    Mint ans = 0;

    F(i, 0, S(vs)) { ans += vs[i] * Mint(b).pow(i); }

    return ans;
  };
```

```
  return { solve(solve, 31), solve(solve, 39) };
};
```

7.10. Centroid Decomposition

`ancestors[u]` returns the list of ancestors of `u` (along with the distance) as we go up the centroid decomposition tree.

```
int main() {
  // WARN: assumes one-indexed nodes
  cin >> n;
  V adj(n + 1, V());

  F(_, 0, n - 1) {
    int u, v;
    cin >> u >> v;
    adj[u].P(v);
    adj[v].P(u);
  }

  V seen(n + 1), subt(n + 1, 0);
  V ancestors(n + 1, V<ii>());

  auto build_subt = [&](auto self, int u, int pu) -> int {
    subt[u] = 1;
    for (auto v : adj[u]) {
      if (v == pu || seen[v]) continue;
      subt[u] += self(self, v, u);
    }
    return subt[u];
  };

  // WARN: returns one centroid only (there might be two)
  auto get_centroid = [&](auto self, int u, int pu, int tree_size) -
> int {
    for (auto v : adj[u]) {
      if (v == pu || seen[v]) continue;
      if (2 * subt[v] > tree_size) return self(self, v, u,
tree_size);
    }
    return u;
  };

  auto build_ancestor
      = [&](auto self, int u, int pu, int centroid, int cur_d = 1) -
> void {
    for (auto v : adj[u]) {
      if (v == pu || seen[v]) continue;
      self(self, v, u, centroid, cur_d + 1);
    }
    ancestors[u].P({ centroid, cur_d });
  };

  auto centroid_decomp = [&](auto self, int init = 1) -> void {
    int centroid
        = get_centroid(get_centroid, init, 0, build_subt(build_subt,
init, 0));
    for (auto v : adj[centroid]) {
      if (seen[v]) continue;
      build_ancestor(build_ancestor, v, centroid, centroid);
    }
    seen[centroid] = true;
    for (auto v : adj[centroid]) {
      if (seen[v]) continue;
      self(self, v);
    }
  };
}
```

7.11. 2-SAT

```
// WARN: assumes zero-indxed nodes
class two_sat {
  int n;
  vector<vector<int>> ori, rev;

public:
  two_sat(int n)
      : n(n) {
    ori.assign(2 * n, {});
    rev.assign(2 * n, {});
  }
  void add_clause(int u, bool u_pos, int v, bool v_pos) {
    ori[2 * u + u_pos].push_back(2 * v + (!v_pos));
    ori[2 * v + v_pos].push_back(2 * u + (!u_pos));
    rev[2 * v + (!v_pos)].push_back(2 * u + u_pos);
    rev[2 * u + (!u_pos)].push_back(2 * v + v_pos);
  }
  bool is_satisfiable() {
    int color = 1;
    stack<int> stk;
    vector<int> seen(2 * n), colors(2 * n);
    auto build_stk = [&](auto build_stk, int u) -> void {
      seen[u] = true;
      for (auto &v : ori[u])
        if (!seen[v]) {
          build_stk(build_stk, v);
        }
      stk.push(u);
    };
    for (int u = 0; u < 2 * n; u++)
      if (!seen[u]) build_stk(build_stk, u);
    auto build_scc = [&](auto build_scc, int u) -> void {
      colors[u] = color, seen[u] = true;
      for (auto &v : rev[u])
        if (!colors[v]) {
          build_scc(build_scc, v);
        }
    };
    while (!stk.empty()) {
      int u = stk.top();
      stk.pop();
      if (!colors[u]) {
        build_scc(build_scc, u);
        color++;
      }
    }
    for (int i = 0; i < n; i++) {
      if (colors[2 * i] == colors[2 * i + 1]) return false;
    }
    return true;
  }
};
```

7.12. Small-To-Large Merging

Instead, of each node having its own set of objects, try to reuse the children sets as much as possible. This reduces the number of swaps/inserts and improves the constant factor. The following snippet solves CSES's Distinct Colors, which asks you to find the number of distinct colors in each subtree.

```
void solve_t() {
  cin >> n;

  vector<set<int>> col(n + 1);
  vector adj(n + 1, vector<int>());
  vector ans(n + 1, 0);
  vector c(n + 1, 0);

  G(u, 1, n) { cin >> c[u]; }

  F(_, 0, n - 1) {
    int u, v;
    cin >> u >> v;
    adj[u].P(v);
    adj[v].P(u);
  }

  // auto merge = [&](set<int> &s, set<int> &t) -> void {
  //   if (S(s) < S(t)) swap(s, t);
  //   for (auto &c : t) s.I(c);
  //   t.clear();
  // };

  auto solve = [&](auto self, int u = 1, int pu = 0) -> set<int> * {
    vector<set<int> *> ptrs;

    for (auto v : adj[u]) {
      if (v == pu) continue;
      ptrs.P(self(self, v, u));
    }

    if (empty(ptrs)) {
      col[u].I(c[u]), ans[u] = 1;
      return &col[u];
    }

    int mxi = 0, mxv = S(*ptrs[0]);

    F(i, 1, S(ptrs)) {
      if (S(*ptrs[i]) > mxv) {
        mxv = S(*ptrs[i]);
        mxi = i;
      }
    }

    F(i, 0, S(ptrs)) {
      if (i == mxi) continue;
      for (auto x : *ptrs[i]) {
        ptrs[mxi]->insert(x);
      }
      ptrs[i]->clear();
    }

    ptrs[mxi]->insert(c[u]);
    ans[u] = S(*ptrs[mxi]);

    return ptrs[mxi];
  };
  solve(solve);

  G(u, 1, n) { cout << ans[u] << N(u, n); }
}
```

8. Flows

8.1. Edmonds-Karp

Suppose we have already set up `adj` and `cap`, where `cap` represents the capacity of each edge; we may use this algorithm to determine the maximum flow in a flow network. When adding an edge, we need to include both (u, v) and (v, u) in `adj`, however, we'll only set the capacity of (u, v) to whatever it is.

To determine the **minimum cut**, find all vertices that can be reached from the source, using only edges with positive residual capacity.

```
int main() {
  V adj(n, V()), cap(n, V(n, 0));

  auto bfs = [&](int s, int t, vector<int> &par) -> int {
    par.assign(ssize(adj), -1), par[s] = -2;
    queue<ii> q;
    q.push({ s, oo });
    while (!empty(q)) {
      auto [u, flow] = q.front();
      q.pop();
      for (auto &v : adj[u]) {
        if (par[v] != -1 || cap[u][v] == 0) continue;
        par[v]       = u;
```

```
        int new_flow = min(flow, cap[u][v]);
        if (v == t) return new_flow;
        q.push({ v, new_flow });
      }
    }
    return 0;
  };

  // WARN: you may need to alter `int` to `ll` depending on the max flow
  auto max_flow = [&](int s, int t) -> int {
    int flow = 0, new_flow = 0;
    vector par(ssize(adj), 0);
    while ((new_flow = bfs(s, t, par))) {
      flow += new_flow;
      int u = t;
      while (u != s) {
        int pu = par[u];
        cap[pu][u] -= new_flow, cap[u][pu] += new_flow;
        u = pu;
      }
    }
    return flow;
  };
}
```

8.2. Dinic's Algorithm

Remember to reset all the edges if you need to compute the flow to some new destination. It's also possible to get the incremental flow by adding edges and calling `flow` repeatedly.

```
struct FlowEdge {
  int v, u;
  ll cap, flow = 0;
  FlowEdge(int v, int u, ll cap)
      : v(v)
      , u(u)
      , cap(cap) {}
};

struct Dinic {
  const ll flow_inf = 00;
  vector<FlowEdge> edges;
  vector<vector<int>> adj;

  int n, m = 0;
  int s, t;

  vector<int> level, ptr;
  queue<int> q;

  Dinic(int n, int s)
      : n(n)
      , s(s) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
  }

  void add_edge(int v, int u, ll cap) { // NOTE: add edge (v -> u)
    edges.emplace_back(v, u, cap);
    edges.emplace_back(u, v, 0);
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
  }

  bool bfs() {
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      for (int id : adj[v]) {
        if (edges[id].cap == edges[id].flow) continue;
        if (level[edges[id].u] != -1) continue;
        level[edges[id].u] = level[v] + 1;
        q.push(edges[id].u);
      }
    }
    return level[t] != -1;
  }

  ll dfs(int v, ll pushed) {
    if (pushed == 0) return 0;
    if (v == t) return pushed;
    for (int &cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
      int id = adj[v][cid];
      int u  = edges[id].u;
      if (level[v] + 1 != level[u]) continue;
      ll tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
      if (tr == 0) continue;
      edges[id].flow += tr;
      edges[id ^ 1].flow -= tr;
      return tr;
    }
    return 0;
  }

  ll flow(int t_) {
    t    = t_;
    ll f = 0;
    while (true) {
      fill(A(level), -1);
      level[s] = 0;
      q.push(s);
      if (!bfs()) break;
      fill(A(ptr), 0);
      while (ll pushed = dfs(s, flow_inf)) {
        f += pushed;
      }
    }
    return f;
  }
};
```

9. Sparse Table

```
template <typename T>
class SparseTable {
  const int K = 25;
  int n;
  vector<vector<T>> st;
  T conquer(T x, T y) { return min(x, y); } // WARN: rmb to tweak
public:
  SparseTable(vector<T> &a)
      : n(S(a))
      , st(K + 1, vector<T>(n)) {
    copy(all(a), begin(st[0]));
    for (int i = 1; i <= K; i++) {
      for (int j = 0; j + (1 << i) <= n; j++) {
        st[i][j] = conquer(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
      }
    }
  }
  T query(int l, int r) {
    int i = __lg(r - l + 1);
    return conquer(st[i][l], st[i][r - (1 << i) + 1]);
  }
};
```

10. Fenwick Tree

10.1. One-Dimensional Queries

```
// NOTE: assumes one-indexed queries
template <typename T = int>
class FenwickTree {
  int n;
  vector<T> tree;

public:
  FenwickTree(int tree_size)
      : n(tree_size) {
    tree.assign(n + 1, {});
  }
  void point_add(int i, T addend) {
    for (; i <= n; i += lso(i)) tree[i] += addend;
  }
  void range_add(int l, int r, T addend) {
    point_add(l, addend);
    point_add(r + 1, -addend);
  }
  T point_ask(int i) {
    T sum = {};
    for (; i; i -= lso(i)) sum += tree[i];
    return sum;
  }
  T range_ask(int x, int y) { return point_ask(y) - point_ask(x - 1); }
};
```

10.2. Two-Dimensional Queries

```
// NOTE: assumes one-indexed queries
template <typename T = int>
class FenwickTree {
  int height;
  int width;
  vector<vector<T>> tree;

public:
  FenwickTree(int h, int w)
      : height(h)
      , width(w) {
    tree.assign(height + 1, vector<T>(width + 1));
  }
  T point_ask(int x, int y) {
    T ans = {};
    for (; x; x -= lso(x)) {
      for (int i = y; i; i -= lso(i)) {
        ans += tree[x][i];
      }
    }
    return ans;
  }
  T area_ask(int x1, int y1, int x2, int y2) {
    T ans = point_ask(x2, y2) + point_ask(x1 - 1, y1 - 1);
    ans -= point_ask(x2, y1 - 1) + point_ask(x1 - 1, y2);
    return ans;
  }
  void point_add(int x, int y, T addend) {
    for (; x <= height; x += lso(x)) {
      for (int i = y; i <= width; i += lso(i)) {
        tree[x][i] += addend;
      }
    }
  }
  void area_add(int x1, int y1, int x2, int y2, T addend) {
    point_add(x1, y1, addend);
    point_add(x2 + 1, y2 + 1, addend);
    point_add(x2 + 1, y1, -addend);
    point_add(x1, y2 + 1, -addend);
  }
};
```

10.3. Range Addition/Queries

This is a tiny bit faster compared to using segment trees.

```cpp
// NOTE: assumes one-indexed queries
template <typename T = ll>
class FenwickTree {
  int n;
  vector<T> tree;

public:
  FenwickTree(int tree_size)
      : n(tree_size) {
    tree.assign(n + 1, {});
  }
  void point_add(int i, T addend) {
    for (; i <= n; i += lso(i)) tree[i] += addend;
  }
  T point_ask(int i) {
    T sum = {};
    for (; i; i -= lso(i)) sum += tree[i];
    return sum;
  }
};
class DoubleFenwick {
  int n;
  FenwickTree<ll> b1, b2;
  ll prefix_sum(int idx) { return b1.point_ask(idx) * idx - b2.point_ask(idx); }

public:
  DoubleFenwick(int n)
      : n(n)
      , b1(n)
      , b2(n) {}
  void range_add(int l, int r, ll x) {
    b1.point_add(l, x);
    b1.point_add(r + 1, -x);
    b2.point_add(l, x * (l - 1));
    b2.point_add(r + 1, -x * r);
  }
  ll range_query(int l, int r) { return prefix_sum(r) - prefix_sum(l - 1); }
};
```

11. Segment Tree

11.1. RMQ (Iterative)

It can help to avoid TLE in certain situations.

```cpp
template <typename T>
class SegTree {
  int n;
  vector<T> a, tree;
  void build() {
    for (int i = n - 1; i >= 1; --i) {
      tree[i] = conquer(tree[i << 1], tree[(i << 1) ^ 1]);
    }
  }

public:
  SegTree(vector<T> &a)
      : a(a) {
    n = ssize(a);
    tree.assign(2 * n, {});
    for (int i = 0; i < n; i++) tree[i + n] = a[i];
    build();
  }
  T conquer(T x, T y) { // WARN: rmb to tweak
    return min(x, y);
  }
  void update(int i, T val) {
    for (tree[i += n] = val; i > 1; i >>= 1) {
      tree[i >> 1] = conquer(tree[i], tree[i ^ 1]);
    }
  }
  T query(int l, int r) {
    T ans = numeric_limits<T>::max(); // WARN: depends on `conquer`
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1) ans = conquer(ans, tree[l++]);
      if (r & 1) ans = conquer(ans, tree[--r]);
    }
    return ans;
  }
};
```

11.2. RMQ (Recursive)

```cpp
template <typename T>
class SegTree {
  int n;
  vector<T> a, tree;
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l];
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void update(int i, int tl, int tr, int pos, T val) {
    if (tl == tr) tree[i] = val;
    else {
      int tm = (tl + tr) / 2;
      if (pos <= tm) update(2 * i, tl, tm, pos, val);
      else update(2 * i + 1, tm + 1, tr, pos, val);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  T query(int i, int tl, int tr, int l, int r) {
    if (l > r) return numeric_limits<T>::max(); // WARN: depends on `conquer`
    if (l == tl && r == tr) return tree[i];
    int tm = (tl + tr) / 2;
    return conquer(
        query(2 * i, tl, tm, l, min(r, tm)),
        query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r));
  }

public:
  SegTree(vector<T> &a)
      : a(a) {
    n = ssize(a);
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  T conquer(T x, T y) { // WARN: rmb to tweak
    return min(x, y);
  }
  void update(int pos, T val) { update(1, 0, n - 1, pos, val); }
  T query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

11.3. Lazy Propagation

11.3.1. Range Addition/Assignment (Min Query)

```cpp
template <typename T>
class LazyTree {
  struct LazyNode {
    T assign_val;
    T add_val;
    bool has_assign;
    bool has_add;
    bool has_update() { return has_assign || has_add; }
  };
  int n;
  vector<LazyNode> lazy;
  vector<T> a, tree;
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l];
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void push(int i) {
    if (lazy[i].has_assign) {
      tree[2 * i]         = lazy[i].assign_val;
      tree[2 * i + 1]     = lazy[i].assign_val;
      lazy[2 * i].add_val = lazy[2 * i + 1].add_val = {};
      lazy[2 * i].has_add = lazy[2 * i + 1].has_add = false;
      lazy[2 * i].assign_val = lazy[2 * i + 1].assign_val = lazy[i].assign_val;
      lazy[2 * i].has_assign = lazy[2 * i + 1].has_assign = true;
    } else {
      if (lazy[2 * i].has_assign) {
        tree[2 * i] += lazy[i].add_val;
        lazy[2 * i].assign_val += lazy[i].add_val;
      } else {
        tree[2 * i] += lazy[i].add_val;
        lazy[2 * i].add_val += lazy[i].add_val;
        lazy[2 * i].has_add = true;
      }
      if (lazy[2 * i + 1].has_assign) {
        tree[2 * i + 1] += lazy[i].add_val;
        lazy[2 * i + 1].assign_val += lazy[i].add_val;
      } else {
        tree[2 * i + 1] += lazy[i].add_val;
        lazy[2 * i + 1].add_val += lazy[i].add_val;
        lazy[2 * i + 1].has_add = true;
      }
    }
    lazy[i].has_assign = lazy[i].has_add = false;
    lazy[i].assign_val = lazy[i].add_val = {};
  }
  void add(int i, int tl, int tr, int l, int r, T addend) {
    if (l > r) return;
    if (l == tl && r == tr) {
      if (lazy[i].has_assign) {
        tree[i] += addend;
        lazy[i].assign_val += addend;
      } else {
        tree[i] += addend;
        lazy[i].add_val += addend;
        lazy[i].has_add = true;
      }
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      add(2 * i, tl, tm, l, min(r, tm), addend);
      add(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, addend);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void assign(int i, int tl, int tr, int l, int r, T val) {
    if (l > r) return;
    if (l == tl && r == tr) {
      if (lazy[i].has_add) {
        lazy[i].has_add = false;
        lazy[i].add_val = {};
      }
      tree[i]             = val;
      lazy[i].assign_val = val;
      lazy[i].has_assign = true;
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      assign(2 * i, tl, tm, l, min(r, tm), val);
      assign(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, val);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
```

```
  T query(int i, int tl, int tr, int l, int r) {
    if (l > r) return numeric_limits<T>::max(); // WARN: depends on `conquer`
    if (l == tl && r == tr) return tree[i];
    if (lazy[i].has_update()) push(i);
    int tm = (tl + tr) / 2;
    return conquer(
        query(2 * i, tl, tm, l, min(r, tm)),
        query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r));
  }

public:
  LazyTree(vector<T> &a)
      : a(a) {
    n = ssize(a);
    lazy.assign(4 * n, {});
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  T conquer(T x, T y) { // WARN: rmb to tweak
    return min(x, y);
  }
  void add(int l, int r, T addend) { add(1, 0, n - 1, l, r, addend); }
  void assign(int l, int r, T val) { assign(1, 0, n - 1, l, r, val); }
  T query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

11.3.2. Range Addition/Assignment (Sum Query)

```
template <typename T>
class LazyTree {
  struct LazyNode {
    T assign_val;
    T add_val;
    bool has_assign;
    bool has_add;
    bool has_update() { return has_assign || has_add; }
  };
  int n;
  vector<LazyNode> lazy;
  vector<ll> s;
  vector<T> a, tree;
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l], s[i] = 1;
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      s[i]    = (r - l + 1);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void push(int i) {
    if (lazy[i].has_assign) {
      tree[2 * i]         = s[2 * i] * lazy[i].assign_val;
      tree[2 * i + 1]     = s[2 * i + 1] * lazy[i].assign_val;
      lazy[2 * i].add_val = lazy[2 * i + 1].add_val = {};
      lazy[2 * i].has_add = lazy[2 * i + 1].has_add = false;
      lazy[2 * i].assign_val = lazy[2 * i + 1].assign_val = lazy[i].assign_val;
      lazy[2 * i].has_assign = lazy[2 * i + 1].has_assign = true;
    } else {
      if (lazy[2 * i].has_assign) {
        tree[2 * i] += s[2 * i] * lazy[i].add_val;
        lazy[2 * i].assign_val += lazy[i].add_val;
      } else {
        tree[2 * i] += s[2 * i] * lazy[i].add_val;
        lazy[2 * i].add_val += lazy[i].add_val;
        lazy[2 * i].has_add = true;
      }
      if (lazy[2 * i + 1].has_assign) {
        tree[2 * i + 1] += s[2 * i + 1] * lazy[i].add_val;
        lazy[2 * i + 1].assign_val += lazy[i].add_val;
        lazy[2 * i + 1].has_assign = true;
      } else {
        tree[2 * i + 1] += s[2 * i + 1] * lazy[i].add_val;
        lazy[2 * i + 1].add_val += lazy[i].add_val;
        lazy[2 * i + 1].has_add = true;
      }
    }
    lazy[i].has_assign = lazy[i].has_add = false;
    lazy[i].assign_val = lazy[i].add_val = {};
  }
  void add(int i, int tl, int tr, int l, int r, T addend) {
    if (l > r) return;
    if (l == tl && r == tr) {
      if (lazy[i].has_assign) {
        tree[i] += s[i] * addend;
        lazy[i].assign_val += addend;
      } else {
        tree[i] += s[i] * addend;
        lazy[i].add_val += addend;
        lazy[i].has_add = true;
      }
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      add(2 * i, tl, tm, l, min(r, tm), addend);
      add(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, addend);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void assign(int i, int tl, int tr, int l, int r, T val) {
    if (l > r) return;
    if (l == tl && r == tr) {
      if (lazy[i].has_add) {
        lazy[i].has_add = false;
        lazy[i].add_val = {};
      }
      tree[i]            = s[i] * val;
      lazy[i].assign_val = val;
      lazy[i].has_assign = true;
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      assign(2 * i, tl, tm, l, min(r, tm), val);
      assign(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, val);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  T query(int i, int tl, int tr, int l, int r) {
    if (l > r) return {}; // WARN: depends on `conquer`
    if (l == tl && r == tr) return tree[i];
    if (lazy[i].has_update()) push(i);
    int tm = (tl + tr) / 2;
    return conquer(
        query(2 * i, tl, tm, l, min(r, tm)),
        query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r));
  }

public:
  LazyTree(vector<T> &a)
      : a(a) {
    n = ssize(a);
    s.assign(4 * n, 0);
    lazy.assign(4 * n, {});
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  T conquer(T x, T y) { // WARN: rmb to tweak
    return x + y;
  }
  void add(int l, int r, T addend) { add(1, 0, n - 1, l, r, addend); }
  void assign(int l, int r, T val) { assign(1, 0, n - 1, l, r, val); }
  T query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

11.3.3. Range Addition/Multiplication (Sum Query)

```
template <typename T>
class LazyTree {
  struct LazyNode {
    T multiply_val = Mint(1);
    T add_val;
    bool has_multiply;
    bool has_add;
    bool has_update() { return has_multiply || has_add; }
  };
  int n;
  vector<LazyNode> lazy;
  vector<Mint> s;
  vector<T> a, tree;
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l], s[i] = 1;
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      s[i]    = (r - l + 1);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void push(int i) {
    if (i >= 4 * n) return;
    if (2 * i < 4 * n) {
      tree[2 * i] *= lazy[i].multiply_val;
      tree[2 * i] += lazy[i].add_val;
      lazy[2 * i].multiply_val *= lazy[i].multiply_val;
      lazy[2 * i].add_val *= lazy[i].multiply_val;
      lazy[2 * i].add_val += lazy[i].add_val;
      lazy[2 * i].has_multiply = lazy[2 * i].has_add = true;
    }
    if (2 * i + 1 < 4 * n) {
      tree[2 * i + 1] *= lazy[i].multiply_val;
      tree[2 * i + 1] += lazy[i].add_val;
      lazy[2 * i + 1].multiply_val *= lazy[i].multiply_val;
      lazy[2 * i + 1].add_val *= lazy[i].multiply_val;
      lazy[2 * i + 1].add_val += lazy[i].add_val;
      lazy[2 * i + 1].has_multiply = lazy[2 * i + 1].has_add = true;
    }
    lazy[i].has_multiply = lazy[i].has_add = false;
    lazy[i].multiply_val = Mint(1), lazy[i].add_val = {};
  }
  void add(int i, int tl, int tr, int l, int r, T addend) {
    if (l > r) return;
    if (l == tl && r == tr) {
      tree[i] += s[i] * addend;
      lazy[i].add_val += addend;
      lazy[i].has_add = true;
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      add(2 * i, tl, tm, l, min(r, tm), addend);
      add(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, addend);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void multiply(int i, int tl, int tr, int l, int r, T val) {
    if (l > r) return;
    if (l == tl && r == tr) {
      tree[i] *= val;
      lazy[i].add_val *= val, lazy[i].multiply_val *= val;
      lazy[i].has_multiply = true;
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      multiply(2 * i, tl, tm, l, min(r, tm), val);
      multiply(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, val);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  T query(int i, int tl, int tr, int l, int r) {
    if (l > r) return {}; // WARN: depends on `conquer`
```

```
    if (l == tl && r == tr) return tree[i];
    if (lazy[i].has_update()) push(i);
    int tm = (tl + tr) / 2;
    return conquer(
        query(2 * i, tl, tm, l, min(r, tm)),
        query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r));
  }

public:
  LazyTree(vector<T> &a)
      : a(a) {
    n = ssize(a);
    s.assign(4 * n, 0);
    lazy.assign(4 * n, {});
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  T conquer(T x, T y) { // WARN: rmb to tweak
    return x + y;
  }
  void add(int l, int r, T addend) { add(1, 0, n - 1, l, r, addend); }
  void multiply(int l, int r, T val) { multiply(1, 0, n - 1, l, r, val); }
  T query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

11.3.4. Polynomial Queries

Add `[base, base+1, base+2, ...]` to the range `[l, r]`.

```
class LazyTree {
  int n;
  vector<ll> s, lazy, d;
  vector<ll> a, tree;
  ll sum(ll base, ll diff, ll len) {
    return (2 * base + diff * (len - 1)) * len / 2;
  }
  ll conquer(ll x, ll y) { return x + y; } // WARN: rmb to tweak
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l], s[i] = 1;
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      s[i]    = (r - l + 1);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void push(int i) {
    tree[2 * i] += sum(lazy[i], d[i], s[2 * i]);
    tree[2 * i + 1] += sum(s[2 * i] * d[i] + lazy[i], d[i], s[2 * i + 1]);
    lazy[2 * i] += lazy[i], d[2 * i] += d[i];
    lazy[2 * i + 1] += s[2 * i] * d[i] + lazy[i], d[2 * i + 1] += d[i];
    lazy[i] = d[i] = 0;
  }
  void
  add(int i, int tl, int tr, int l, int r,
      ll base) { // WARN: requires `base` ≥ 1
    if (l > r) return;
    if (l == tl && r == tr) {
      tree[i] += sum(base, 1, r - l + 1);
      lazy[i] += base;
      d[i]++;
    } else {
      if (lazy[i]) push(i);
      int tm = (tl + tr) / 2, gap = max(0, min(r, tm) - l + 1);
      add(2 * i, tl, tm, l, min(r, tm), base);
      add(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, base + gap);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  ll query(int i, int tl, int tr, int l, int r) {
    if (l > r) return {}; // WARN: depends on `conquer`
    if (l == tl && r == tr) return tree[i];
    if (lazy[i]) push(i);
    int tm = (tl + tr) / 2;
    return conquer(
        query(2 * i, tl, tm, l, min(r, tm)),
        query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r));
  }

public:
  LazyTree(vector<ll> &a)
      : a(a) {
    n = ssize(a);
    s.assign(4 * n, 0);
    lazy.assign(4 * n, {});
    d.assign(4 * n, {});
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  void add(int l, int r, ll base) { add(1, 0, n - 1, l, r, base); }
  ll query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

11.3.5. Range Addition (Count Minimums)

`query` returns the minimum value in a range, as well as the corresponding count.

```
class LazyTree {
  struct LazyNode {
    int add_val;
    bool has_add;
    bool has_update() { return has_add; }
  };
  int n;
  vector<LazyNode> lazy;
  vector<ii> a, tree;
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l];
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void push(int i) {
    tree[2 * i][0] += lazy[i].add_val;
    lazy[2 * i].add_val += lazy[i].add_val;
    lazy[2 * i].has_add = true;
    tree[2 * i + 1][0] += lazy[i].add_val;
    lazy[2 * i + 1].add_val += lazy[i].add_val;
    lazy[2 * i + 1].has_add = true;
    lazy[i].has_add         = false;
    lazy[i].add_val         = {};
  }
  void add(int i, int tl, int tr, int l, int r, int addend) {
    if (l > r) return;
    if (l == tl && r == tr) {
      tree[i][0] += addend;
      lazy[i].add_val += addend;
      lazy[i].has_add = true;
    } else {
      if (lazy[i].has_update()) push(i);
      int tm = (tl + tr) / 2;
      add(2 * i, tl, tm, l, min(r, tm), addend);
      add(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, addend);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  ii query(int i, int tl, int tr, int l, int r) {
    if (l > r) return { oo, 0 }; // WARN: depends on `conquer`
    if (l == tl && r == tr) return tree[i];
    if (lazy[i].has_update()) push(i);
    int tm = (tl + tr) / 2;
    return conquer(
        query(2 * i, tl, tm, l, min(r, tm)),
        query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r));
  }

public:
  LazyTree(vector<ii> &a)
      : a(a) {
    n = ssize(a);
    lazy.assign(4 * n, {});
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  ii conquer(ii x, ii y) { // WARN: rmb to tweak
    if (x[0] == y[0]) return { x[0], x[1] + y[1] };
    return min(x, y);
  }
  void add(int l, int r, int addend) { add(1, 0, n - 1, l, r, addend); }
  ii query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

11.4. Persistent Segment Tree

Stores earlier versions of the tree. To create a new version, use `cpy_root`.

```
template <typename T>
class PersistentTree {
  struct Node {
    Node *l, *r;
    T s;
    Node(T s)
        : l(nullptr)
        , r(nullptr)
        , s(s) {}
    Node(Node *u)
        : l(u->l)
        , r(u->r)
        , s(u->s) {}
    Node(Node *l, Node *r)
        : l(l)
        , r(r)
        , s() { // WARN: depends on `conquer`
      if (!l && r) s = r->s;
      if (l && !r) s = l->s;
      if (l && r) s = conquer(l->s, r->s);
    }
  };
  int n = 0;
  vector<Node *> roots;
  Node *build(const vector<T> &a, int tl, int tr) {
    if (tl == tr) return new Node(a[tl]);
    int tm = (tl + tr) / 2;
    return new Node(build(a, tl, tm), build(a, tm + 1, tr));
  }
  T query(Node *u, int tl, int tr, int l, int r) {
    if (l > r) return {}; // WARN: depends on `conquer`
    if (l == tl && r == tr) return u->s;
    int tm = (tl + tr) / 2;
    T ls   = query(u->l, tl, tm, l, min(r, tm));
    T rs   = query(u->r, tm + 1, tr, max(l, tm + 1), r);
    return conquer(ls, rs);
  }
  Node *update(Node *u, int tl, int tr, int pos, T s) {
    if (tl == tr) return new Node(s);
    int tm = (tl + tr) / 2;
    if (pos <= tm) return new Node(update(u->l, tl, tm, pos, s), u->r);
    else return new Node(u->l, update(u->r, tm + 1, tr, pos, s));
  }

public:
  PersistentTree(vector<T> &a, int root_cnt, int init)
      : roots(root_cnt) {
    n           = ssize(a);
```

```
    roots[init] = build(a, 0, n - 1);
  }
  static T conquer(T x, T y) { // WARN: rmb to tweak
    return x + y;
  }
  void update(int root, int pos, T s) {
    roots[root] = update(roots[root], 0, n - 1, pos, s);
  }
  void cpy_root(int src, int dst) { roots[dst] = new Node(roots[src]); }
  T query(int root, int l, int r) { return query(roots[root], 0, n - 1, l, r); }
};

int n, q, qtype, root_cnt = 0, QMAX = 100'000;

int main() {
  cin.tie(nullptr)->sync_with_stdio(false);

  cin >> n;
  V a(n + 1, 0LL);

  G(i, 1, n) cin >> a[i];
  PersistentTree ptree(a, QMAX + 1, root_cnt);

  cin >> q;

  while (q--) {
    cin >> qtype;
    if (qtype == 1) {
      int idx, v;
      cin >> idx >> v, root_cnt++;
      ptree.cpy_root(root_cnt - 1, root_cnt);
      ptree.update(root_cnt, idx, v);
    } else {
      int hist_num, l, r;
      cin >> hist_num >> l >> r;
      cout << ptree.query(hist_num, l, r) << '\n';
    }
  }
  return 0;
}
```

11.5. Merge Sort Tree

Stores the result of merge sort operations; may be used to answer queries such as finding the number of items $\leq x$ within a range.

```
struct Data {
  vector<int> raw;
};
template <typename T = Data>
class MergeSortTree {
  int n;
  vector<T> a, tree;
  void build(int i, int l, int r) {
    if (l == r) tree[i] = a[l];
    else {
      int m = (l + r) / 2;
      build(2 * i, l, m);
      build(2 * i + 1, m + 1, r);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  void update(int i, int tl, int tr, int pos, T val) {
    if (tl == tr) tree[i] = val;
    else {
      int tm = (tl + tr) / 2;
      if (pos <= tm) update(2 * i, tl, tm, pos, val);
      else update(2 * i + 1, tm + 1, tr, pos, val);
      tree[i] = conquer(tree[2 * i], tree[2 * i + 1]);
    }
  }
  int query(int i, int tl, int tr, int l, int r, int x) {
    if (l > r) return {}; // WARN: depends on `conquer`
    if (l == tl && r == tr) {
      int pos = upper_bound(A(tree[i].raw), x) - begin(tree[i].raw);
      return pos;
    }
    int tm   = (tl + tr) / 2;
    int lans = query(2 * i, tl, tm, l, min(r, tm), x);
    int rans = query(2 * i + 1, tm + 1, tr, max(l, tm + 1), r, x);
    return lans + rans;
  }

public:
  MergeSortTree(vector<T> &a)
      : a(a) {
    n = ssize(a);
    tree.assign(4 * n, {});
    build(1, 0, n - 1);
  }
  T conquer(Data x, Data y) { // WARN: rmb to tweak
    Data ans;
    merge(A(x.raw), A(y.raw), back_inserter(ans.raw));
    return ans;
  }
  void update(int pos, T val) { update(1, 0, n - 1, pos, val); }
  int query(int l, int r, int x) { return query(1, 0, n - 1, l, r, x); }
};
```

12. Convolution

12.1. Number Theoretic Transform

```
constexpr ll MOD = 998'244'353, root = 62;
ll modpow(ll b, ll p) {
  if (p == 0) return 1LL;
  ll ans = modpow(b, p / 2);
  ans    = ((__int128)(ans)*ans) % MOD;
  if (p & 1) ans = (ans * b) % MOD;
  return ans;
}

void ntt(vector<ll> &a) {
  int n = ssize(a), L = 31 - __builtin_clz(n);
  static vector<ll> rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = { 1, modpow(root, MOD >> s) };
    for (int i = k; i < 2 * k; i++) rt[i] = rt[i / 2] * z[i & 1] % MOD;
  }
  vector<int> rev(n);
  for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  for (int i = 0; i < n; i++)
    if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      for (int j = 0; j < k; j++) {
        ll z = rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
        a[i + j + k] = ai - z + (z > ai ? MOD : 0);
        ai += (ai + z >= MOD ? z - MOD : z);
      }
}

vector<ll> conv(const vector<ll> &a, const vector<ll> &b) {
  if (a.empty() || b.empty()) return {};
  int s = ssize(a) + ssize(b) - 1, B = 32 - __builtin_clz(s), n = 1 << B;
  int inv = modpow(n, MOD - 2);
  vector<ll> left(a), right(b), out(n);
  left.resize(n), right.resize(n);
  ntt(left), ntt(right);
  for (int i = 0; i < n; i++)
    out[-i & (n - 1)] = (ll)left[i] * right[i] % MOD * inv % MOD;
  ntt(out);
  return { out.begin(), out.begin() + s };
}
```

13. Math

13.1. Extended Euclidean Algorithm

Finds x and y such that $xa + yb = \gcd(a, b)$.

```
class GCD {
  static pair<ll, ll> solve(ll a, ll b) {
    if (b == 0) return { 1, 0 };
    auto [y, x] = solve(b, a % b);
    y -= a / b * x;
    return { x, y };
  }

public:
  static pair<ll, ll> extended_gcd(ll a, ll b) {
    auto [x, y] = solve(a, b);
    if (x * a + b * y == -gcd(a, b)) x *= -1, y *= -1;
    return { x, y };
  }
};
```

13.2. Mint

```
class Mint {
  constexpr ll mod(ll x) const {
    if (x < 0) x += MOD;
    return x >= MOD ? x % MOD : x;
  }

public:
  static constexpr ll MOD = 1'000'000'007; // WARN: rmb to tweak
  ll v;
  constexpr ll pow(ll p) const {
    if (!p) return 1;
    ll ans = pow(p / 2);
    return mod(p & 1 ? v * mod(ans * ans) : mod(ans * ans));
  }
  constexpr ll i() const { return pow(MOD - 2); }
  constexpr Mint(ll v = 0)
      : v(mod(v)) {}
  constexpr Mint(const Mint &x)
      : v(x.v) {}
  constexpr Mint &operator++() {
    v = mod(v + 1);
    return *this;
  }
  constexpr Mint operator++(int) {
    Mint _ = *this;
    ++(*this);
    return _;
  }
  constexpr Mint &operator--() {
    v = mod(v - 1);
    return *this;
  }
  constexpr Mint operator--(int) {
    Mint _ = *this;
    --(*this);
    return _;
  }
  constexpr Mint &operator=(const Mint &y) {
    v = y.v;
    return *this;
  }
  constexpr Mint &operator+=(const Mint &y) {
    v = mod(v + y.v);
    return *this;
```

```
  }
  constexpr Mint &operator-=(const Mint &y) {
    v = mod(v - y.v);
    return *this;
  }
  constexpr Mint &operator*=(const Mint &y) {
    v = mod(v * y.v);
    return *this;
  }
  constexpr Mint &operator/=(const Mint &y) {
    v = mod(v * y.i());
    return *this;
  }
  friend constexpr Mint operator+(Mint x, const Mint &y) {
    x += y;
    return x;
  }
  friend constexpr Mint operator-(Mint x, const Mint &y) {
    x -= y;
    return x;
  }
  friend constexpr Mint operator*(Mint x, const Mint &y) {
    x *= y;
    return x;
  }
  friend constexpr Mint operator/(Mint x, const Mint &y) {
    x /= y;
    return x;
  }
  friend constexpr bool operator==(const Mint &x, const Mint &y) {
    return x.v == y.v;
  }
  friend constexpr bool operator!=(const Mint &x, const Mint &y) {
    return x.v != y.v;
  }
  friend ostream &operator<<(ostream &os, const Mint &x) { return os
<< x.v; }
};
```

13.3. Sieve of Eratosthenes

```
constexpr int SIEVE_SIZE = 1'000'000; // WARN: rmb to tweak
bitset<SIEVE_SIZE + 1> bs;
vector<ll> primes;
void sieve() {
  bs.set();
  bs[0] = bs[1] = false;
  for (ll i = 2; i <= SIEVE_SIZE; i++)
    if (bs[i]) {
      for (ll j = i * i; j <= SIEVE_SIZE; j += i) bs[j] = false;
      primes.push_back(i);
    }
}
```

13.3.1. Euler's Totient Function

$\varphi(n)$ computes the positive integers up to n that are coprime to n. We can compute it from the prime factors of n.

```
ll eulerPhi(ll n) {
  ll ans = n;
  for (int i = 0; i < S(primes) && primes[i] * primes[i] <= n; i++) {
    if (n % primes[i] == 0) ans -= ans / primes[i];
    while (n % primes[i] == 0) n /= primes[i];
  }
  return ans != 1 ? ans - ans / n : ans;
}
```

We can also compute it using a modified `sieve`.

```
const int SIEVE_SIZE = 1'000'000; // WARN: rmb to tweak
int eulerPhis[SIEVE_SIZE + 1];
void sieve() {
  for (int i = 1; i <= SIEVE_SIZE; i++) eulerPhis[i] = i;
  for (int i = 2; i <= SIEVE_SIZE; i++)
    if (eulerPhis[i] == i)
      for (int j = i; j <= SIEVE_SIZE; j += i) eulerPhis[j] -=
eulerPhis[j] / i;
}
```

13.4. Fast Factorization

Runs in about $\mathcal{O}\left(n^{\frac{1}{4}}\right)$ time, to be used sparingly.

```
ll modmul(ll x, ll y, ll mod) { return ((__int128)x * y) % mod; }

ll modpow(ll b, ll p, ll mod) {
  if (!p) return 1;
  ll ans = modpow(b, p / 2, mod) % mod;
  ans    = modmul(ans, ans, mod);
  if (p & 1) ans = modmul(ans, b, mod);
  return ans;
}

bool is_prime(ll n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ll A[] = { 2, 325, 9375, 28178, 450775, 9780504, 1795265022 },
     s = __builtin_ctzll(n - 1), d = n >> s;
  for (ll a : A) {
    ll p = modpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--) p = modmul(p, p,
n);
    if (p != n - 1 && i != s) return 0;
  }
  return 1;
}

ll rho(ll n) {
  ll x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ll x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}

vector<ll> factor(ll n) {
  if (n == 1) return { 1 };
  auto solve = [&](auto self, ll n) -> V<ll> {
    if (n == 1) return {};
    if (is_prime(n)) return { n };
    auto d  = rho(n);
    V<ll> a = self(self, d), b = self(self, n / d), ans;
    for (auto x : a) ans.P(x);
    for (auto y : b) ans.P(y);
    return ans;
  };
  auto ans = solve(solve, n);
  return ans;
}
```

13.5. Frobenius Number

The *Frobenius number* is the largest integer that *can't* be expressed as a non-negative linear combination of two coprime integers p and q. It is computed as $pq - p - q$.

13.6. Matrix

May be helpful for matrix multiplication problems.

```
constexpr ll MOD = 1'000'000'007;

template <typename T = ll>
struct Mat2D {
  int n, m;
  vector<vector<T>> g;
  Mat2D(int n, int m, bool identity = false)
      : n(n)
      , m(m)
      , g(n, vector<T>(m)) {
    if (n != m || !identity) return;
    for (int i = 0; i < n; i++) {
      g[i][i] = 1;
    }
  }
  constexpr Mat2D pow(ll p) const {
    if (!p) return Mat2D(this->n, this->m, true);
    Mat2D ans = pow(p / 2);
    return p & 1 ? (*this) * ans * ans : ans * ans;
  }
  constexpr Mat2D &operator*=(const Mat2D &b) {
    assert(this->m == b.n);
    Mat2D c = Mat2D(this->n, b.m);
    for (int i = 0; i < this->n; i++) {
      for (int j = 0; j < b.m; j++) {
        for (int k = 0; k < this->m; k++) {
          c.g[i][j] += this->g[i][k] * b.g[k][j];
          c.g[i][j] %= MOD; // WARN: may not be necessary
        }
      }
    }
    n = c.n, m = c.m, g = c.g;
    return *this;
  }
  friend constexpr Mat2D operator*(Mat2D x, const Mat2D &y) {
    x *= y;
    return x;
  }
};
```

14. Strings

14.1. String Splitting

Because C++ doesn't offer a `split` function by default, we may have to write our own sometimes.

```
auto string_split = [](const string &s, const string &delims) {
  size_t prv = 0, pos;
  vector<string> parsed;
  while ((pos = s.find_first_of(delims, prv)) != string::npos) {
    if (pos > prv) parsed.push_back(s.substr(prv, pos - prv));
    prv = pos + 1;
  }
  if (prv < size(s)) parsed.push_back(s.substr(prv, string::npos));
  return parsed;
};
```

14.2. Hashing

You may use two different prime numbers to create a stronger hash.

```
int NMAX = 500'000; // WARN: rmb to tweak
vector<Mint> pwrs(NMAX), invs(NMAX);

class Shash {
  vector<Mint> p;

public:
  static constexpr Mint pwr = 31;
  static constexpr Mint inv = pwr.i();
  Shash(string s) {
    p.resize(S(s));
    for (int i = 0; i < S(s); i++) {
      p[i] = pwrs[i] * (s[i] - 'a' + 1);
    }
```

```
    partial_sum(begin(p), end(p), begin(p));
  }
  Mint query(int l, int r) {
    if (l == 0) return p[r];
    return (p[r] - p[l - 1]) * invs[l];
  }
};

int main() {
  Mint pw = 1, in = 1;
  for (int i = 0; i < NMAX; i++) {
    pwrs[i] = pw, invs[i] = in;
    pw *= Shash::pwr, in *= Shash::inv;
  }
}
```

14.3. Manacher's Algorithm

Computes the maximum length of a palindrome centered at any index.

```
class Manacher {
  deque<int> p;
  deque<int> manacher_odd(string s) {
    int n = ssize(s);
    s     = '{' + s + '}';
    deque<int> p(n + 2);
    int l = 1, r = 1;
    for (int i = 1; i <= n; i++) {
      p[i] = max(0, min(r - i, p[l + (r - i)]));
      while (s[i - p[i]] == s[i + p[i]]) p[i]++;
      if (i + p[i] > r) l = i - p[i], r = i + p[i];
    }
    p.pop_front(), p.pop_back();
    return p;
  }

public:
  Manacher(const string &s) {
    string t(1, '|');
    for (auto &c : s) t += c, t += '|';
    auto ans = manacher_odd(t);
    ans.pop_front(), ans.pop_back();
    p = ans;
  }
  int get_len(int i, bool is_odd) {
    // NOTE: center palindrome at `i` and `i+1` if `is_odd` is false
    return is_odd ? 2 * (p[2 * i] / 2 - 1) + 1 : p[2 * i + 1] - 1;
  }
};
```

14.4. Z-function

The *Z-function* of a string is an array of length n where the i-th element is equal to the greatest number of characters starting from position i that coincides with the first characters of the string.

```
auto z_function = [](const string &s) -> vector<int> {
  int l = 0, r = 0, n = ssize(s);
  vector<int> z(n);
  for (int i = 1; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  return z;
};
```

14.5. Trie

```
constexpr int K = 26;

struct Vertex {
  vector<int> nxt;
  bool is_end;
  int cnt;
  Vertex()
      : nxt(K, -1)
      , is_end(false)
      , cnt(0) {}
};

vector<Vertex> trie(1);

void insert(const string &s) {
  int u = 0;
  for (const auto &c : s) {
    int i = c - 'a';
    if (trie[u].nxt[i] == -1) {
      trie[u].nxt[i] = ssize(trie);
      trie.emplace_back();
    }
    u = trie[u].nxt[i];
    trie[u].cnt++;
  }
  trie[u].is_end = true;
}

ll search(const string &s) {
  int u  = 0;
  ll ans = 0;
  for (const auto &c : s) {
    int i = c - 'a';
    if (trie[u].nxt[i] == -1) break;
    u = trie[u].nxt[i];
    ans += trie[u].cnt;
  }
  return ans;
}
```

You can also find the maximum XOR of a subarray.

```
class Trie {
  int max_length;
  int node_cnt;
  vector<int> stop;
  vector<vector<int>> trie;

public:
  Trie(int max_length)
      : max_length(max_length)
      , node_cnt(0)
      , stop(max_length + 1, false)
      , trie(max_length, vector<int>(2)) {}
  void insert(int word) {
    int node = 0;
    for (int i = 30; i >= 0; i--) {
      int b = (word >> i) & 1;
      if (!trie[node][b]) trie[node][b] = ++node_cnt;
      node = trie[node][b];
    }
    stop[node] = true;
  }
  int find(int word) {
    int node = 0, s = 0;
    for (int i = 30; i >= 0; i--) {
      int b = (word >> i) & 1;
      if (trie[node][b ^ 1]) {
        if (b ^ 1) s |= (1 << i);
        node = trie[node][b ^ 1];
      } else {
        if (b) s |= (1 << i);
        node = trie[node][b];
      }
    }
    return s;
  }
};
```

14.6. Suffix Array

A *suffix array* contains integers that represent the *starting indexes* of all suffixes of a given string, in sorted order.

```
class SuffixArray {
  static constexpr int alphabet = 256;
  static vector<int> sort_cyclic_shifts(const string &s) {
    int n = ssize(s), classes = 1;
    vector<int> p(n), c(n), pn(n), cn(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++) cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    for (int i = 1; i < n; i++) {
      if (s[p[i]] != s[p[i - 1]]) classes++;
      c[p[i]] = classes - 1;
    }
    for (int h = 0; (1 << h) < n; h++) {
      for (int i = 0; i < n; i++) {
        pn[i] = p[i] - (1 << h);
        if (pn[i] < 0) pn[i] += n;
      }
      fill(begin(cnt), begin(cnt) + classes, 0);
      for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
      for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
      for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
      cn[p[0]] = 0;
      classes  = 1;
      for (int i = 1; i < n; i++) {
        pair<int, int> cur = { c[p[i]], c[(p[i] + (1 << h)) % n] };
        pair<int, int> prv = { c[p[i - 1]], c[(p[i - 1] + (1 << h))
% n] };
        if (cur != prv) classes++;
        cn[p[i]] = classes - 1;
      }
      c.swap(cn);
    }
    return p;
  }

public:
  static vector<int> build(const string &_) {
    string s      = _ + '$';
    vector<int> a = sort_cyclic_shifts(s);
    a.erase(begin(a));
    return a;
  }
};

int n;
string s;
auto sar = SuffixArray::build(s);

auto compare = [](const string &t, int suf_idx) -> int {
  for (int i = 0; i < min((int)ssize(t), n - suf_idx); i++) {
    if (t[i] < s[suf_idx + i]) return -1;
    if (t[i] > s[suf_idx + i]) return 1;
    if (i == ssize(t) - 1) return 0;
  }
  return 1;
};

auto count_substr = [](const string &t) -> int {
  // WARN: `sar` := suffix array (declared in the outer scope)
  int lo = 0, hi = n - 1, mi, an = -1, na = -1;
  while (lo <= hi) {
    mi = midpoint(lo, hi);
    if (compare(t, sar[mi]) > 0) an = mi, lo = mi + 1;
    else hi = mi - 1;
  }
  lo = 0, hi = n - 1;
  while (lo <= hi) {
    mi = midpoint(lo, hi);
    if (compare(t, sar[mi]) >= 0) na = mi, lo = mi + 1;
```

```
    else hi = mi - 1;
  }
  return na - an;
};
```

You can also modify it to retrieve the length of the longest common prefix between two suffixes starting at `i` and `j`.

```
class SuffixArray {
  static constexpr int alphabet = 256;

public:
  static int n;
  static vector<vector<int>> cmat;
  static vector<int> sort_cyclic_shifts(const string &s) {
    n           = ssize(s);
    int classes = 1;
    vector<int> p(n), c(n), pn(n), cn(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++) cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    for (int i = 1; i < n; i++) {
      if (s[p[i]] != s[p[i - 1]]) classes++;
      c[p[i]] = classes - 1;
    }
    cmat.P(c);
    for (int h = 0; (1 << h) < n; h++) {
      for (int i = 0; i < n; i++) {
        pn[i] = p[i] - (1 << h);
        if (pn[i] < 0) pn[i] += n;
      }
      fill(begin(cnt), begin(cnt) + classes, 0);
      for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
      for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
      for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
      cn[p[0]] = 0;
      classes  = 1;
      for (int i = 1; i < n; i++) {
        pair<int, int> cur = { c[p[i]], c[(p[i] + (1 << h)) % n] };
        pair<int, int> prv = { c[p[i - 1]], c[(p[i - 1] + (1 << h))
% n] };
        if (cur != prv) classes++;
        cn[p[i]] = classes - 1;
      }
      c.swap(cn);
      cmat.P(c);
    }
    return p;
  }
  static int get_lcp(int i, int j) { // WARN: doesn't work for `i ==
j`
    int ans = 0;
    for (int k = __lg(n); k >= 0; k--) {
      if (cmat[k][i % n] == cmat[k][j % n]) {
        ans += (1 << k);
        i += (1 << k);
        j += (1 << k);
      }
    }
    return ans;
  }
  static vector<int> build(const string &_) { // NOTE: call at the
start
    string s      = _ + '$';
    vector<int> a = sort_cyclic_shifts(s);
    a.erase(begin(a));
    return a;
  }
};
int SuffixArray::n;
vector<vector<int>> SuffixArray::cmat;
```

14.7. Aho-Corasick Algorithm

This variant checks if any substring of a string `s` matches any string in the dictionary.

```
const int NMAX = 1'000 * 100; // WARN: rmb to tweak

int trie[NMAX + 1][26], fail[NMAX + 1], stop[NMAX + 1], idx;

void insert(const string &s) {
  int i = 0;
  for (auto &c : s) {
    if (!trie[i][c - 'a']) trie[i][c - 'a'] = ++idx;
    i = trie[i][c - 'a'];
  }
  stop[i] = true;
}
void build() { // NOTE: construct suffix link(s)
  queue<int> q;
  for (int i = 0; i < 26; i++)
    if (trie[0][i]) q.push(trie[0][i]);
  while (!empty(q)) {
    int u = q.front();
    q.pop();
    stop[u] |= stop[fail[u]];
    for (int i = 0; i < 26; i++) {
      if (trie[u][i]) {
        fail[trie[u][i]] = trie[fail[u]][i];
        q.push(trie[u][i]);
      } else {
        trie[u][i] = trie[fail[u]][i];
      }
    }
  }
}
bool search(const string &s) {
  int i = 0;
  for (auto &c : s) {
    i = trie[i][c - 'a'];
    if (stop[i]) return true;
  }
  return false;
}
```